

EMPLOYEE WORKFORCE MANAGEMENT & ANALYSIS USING SQL

Project Overview

This project demonstrates the use of SQL for Employee Workforce Management and Analysis. It involves creating an employee dataset and performing various SQL operations such as data retrieval, filtering, aggregation, joins, and window functions.

The analysis helps in understanding employee distribution across departments, salary trends, and experience levels. Advanced SQL features like CASE statements, views, and ranking functions are used to generate meaningful insights.

All query outputs are captured and presented as screenshots to showcase the results and practical implementation of SQL concepts.

Dataset

Dataset: Employee workforce management & analysis

To create an employees table with 15 records representing details such as age, department, designation, salary, and experience.

Key Columns

Column Name	Data Type	Description
employee_id	INT PRIMARY KEY	Unique employee ID
full_name	VARCHAR(100)	Employee full name
Age	INT	Employee age
Gender	VARCHAR(10)	Male/Female
Department	VARCHAR(50)	Department (HR/IT/Finance/Sales/Operations)

Designation	VARCHAR(50)	Job title
Salary	DECIMAL(10,2)	Monthly salary
experience_years	DECIMAL(4,1)	Work experience
Location	VARCHAR(50)	Office location

Data Preparation

Steps performed before analysis:

1. Created a Table Structure

```

1 • create database project;
2 • use project;
3 • create table employees(employee_id INT PRIMARY KEY AUTO_INCREMENT,
4   full_name VARCHAR(100) not null,
5   age INT,
6   gender VARCHAR(10) check(gender in("Male", "Female")),
7   department VARCHAR(50) not null check(department in("HR", "IT", "Finance", "Sales", "Operations")),
8   designation VARCHAR(50) not null check(designation in("HR Associate", "Software Engineer", "Senior Developer", "Sales Executive", "Finance Analyst", "Operat
9   salary DECIMAL(10,2) not null check(salary>0),
10  experience_years DECIMAL(4,1),
11  location VARCHAR(50) not null);
12

```

2. Trigger Before insert created

BEFORE INSERT

employees_BEFORE_INSERT

AFTER INSERT

BEFORE UPDATE

AFTER UPDATE

BEFORE DELETE

AFTER DELETE

```

1 • CREATE DEFINER=`root`@`localhost` TRIGGER `employees_BEFORE_INSERT` BEFORE INSERT ON `employees` FOR EACH ROW BEGIN
2   if New.designation="HR Associate" and not (new.salary between 25000 and 40000) then
3     SIGNAL SQLSTATE '45000'
4     SET MESSAGE_TEXT = 'Salary out of range for HR Associate';
5
6   elseif NEW.designation = "Software Engineer" and not( new.salary between 50000 and 90000) then
7     SIGNAL SQLSTATE '45000'

```

Columns Indexes Foreign Keys Triggers Partitioning Options

Apply Revert

Output

▼ BEFORE INSERT

employees_BEFORE_INSERT

AFTER INSERT

BEFORE UPDATE

AFTER UPDATE

BEFORE DELETE

AFTER DELETE

6

elseif NEW.designation = "Software Engineer" and not(new.salary between 50000 and 90000) then

7

SIGNAL SQLSTATE '45000'

8

SET MESSAGE_TEXT="Salary out of range for Software Engineer";

9

10

elseif new.designation= "Senior Developer" and not(new.salary between 90000 and 140000)then

11

SIGNAL SQLSTATE '45000'

12

SET MESSAGE TEXT="Salary out of range for Senior Developer";

Columns Indexes Foreign Keys Triggers Partitioning Options

Apply Revert

▼ BEFORE INSERT

employees_BEFORE_INSERT

AFTER INSERT

BEFORE UPDATE

AFTER UPDATE

BEFORE DELETE

AFTER DELETE

14

elseif new.designation= "Sales Executive" and not(new.salary between 30000 and 50000)then

15

SIGNAL SQLSTATE '45000'

16

SET MESSAGE_TEXT="Salary out of range for Sales Executive";

17

18

elseif new.designation= "Finance Analyst" and not(new.salary between 45000 and 80000)then

19

SIGNAL SQLSTATE '45000'

20

SET MESSAGE TEXT="Salary out of range for Finance Analyst";

Columns Indexes Foreign Keys Triggers Partitioning Options

Apply Revert

▼ BEFORE INSERT

employees_BEFORE_INSERT

AFTER INSERT

BEFORE UPDATE

AFTER UPDATE

BEFORE DELETE

AFTER DELETE

22

elseif new.designation= "Operations Manager" and not (new.salary between 80000 and 120000) then

23

SIGNAL SQLSTATE '45000'

24

SET MESSAGE_TEXT="Salary out of range for Operations Manager";

25

26

end if;

27

END

Columns Indexes Foreign Keys Triggers Partitioning Options

Apply Revert

3. Inserted 15 details.

```

• insert into employees(full_name, age, gender, department, designation, salary, experience_years, location) values
("Amit Sharma", 28, "Male", "IT", "Software Engineer", 75000.00, 3.5, "Mumbai"),
("Neha Verma", 32, "Female", "HR", "HR Associate", 32000.00, 6.0, "Delhi"),
("Rahul Mehta", 40, "Male", "IT", "Senior Developer", 120000.00, 12.5, "Bangalore"),
("Sneha Kapoor", 26, "Female", "Sales", "Sales Executive", 42000.00, 2.5, "Pune"),
("Arjun Patel", 35, "Male", "Finance", "Finance Analyst", 65000.00, 8.0, "Ahmedabad"),
("Priya Nair", 45, "Female", "Operations", "Operations Manager", 100000.00, 18.0, "Chennai"),
("Vikas Singh", 29, "Male", "IT", "Software Engineer", 68000.00, 4.0, "Hyderabad"),
("Anjali Desai", 31, "Female", "Finance", "Finance Analyst", 72000.00, 7.5, "Mumbai"),
("Karan Malhotra", 38, "Male", "Sales", "Sales Executive", 50000.00, 10.0, "Delhi"),
("Meera Iyer", 42, "Female", "Operations", "Operations Manager", 115000.00, 16.0, "Bangalore"),
("Rohan Gupta", 27, "Male", "IT", "Software Engineer", 82000.00, 5.0, "Noida"),
("Pooja Sharma", 30, "Female", "HR", "HR Associate", 35000.00, 7.0, "Kolkata"),
("Siddharth Jain", 36, "Male", "Finance", "Finance Analyst", 78000.00, 9.0, "Jaipur"),
("Divya Reddy", 33, "Female", "Sales", "Sales Executive", 47000.00, 6.5, "Chandigarh"),
("Manish Kumar", 48, "Male", "Operations", "Operations Manager", 110000.00, 20.0, "Indore");

```

4. For age conditions-

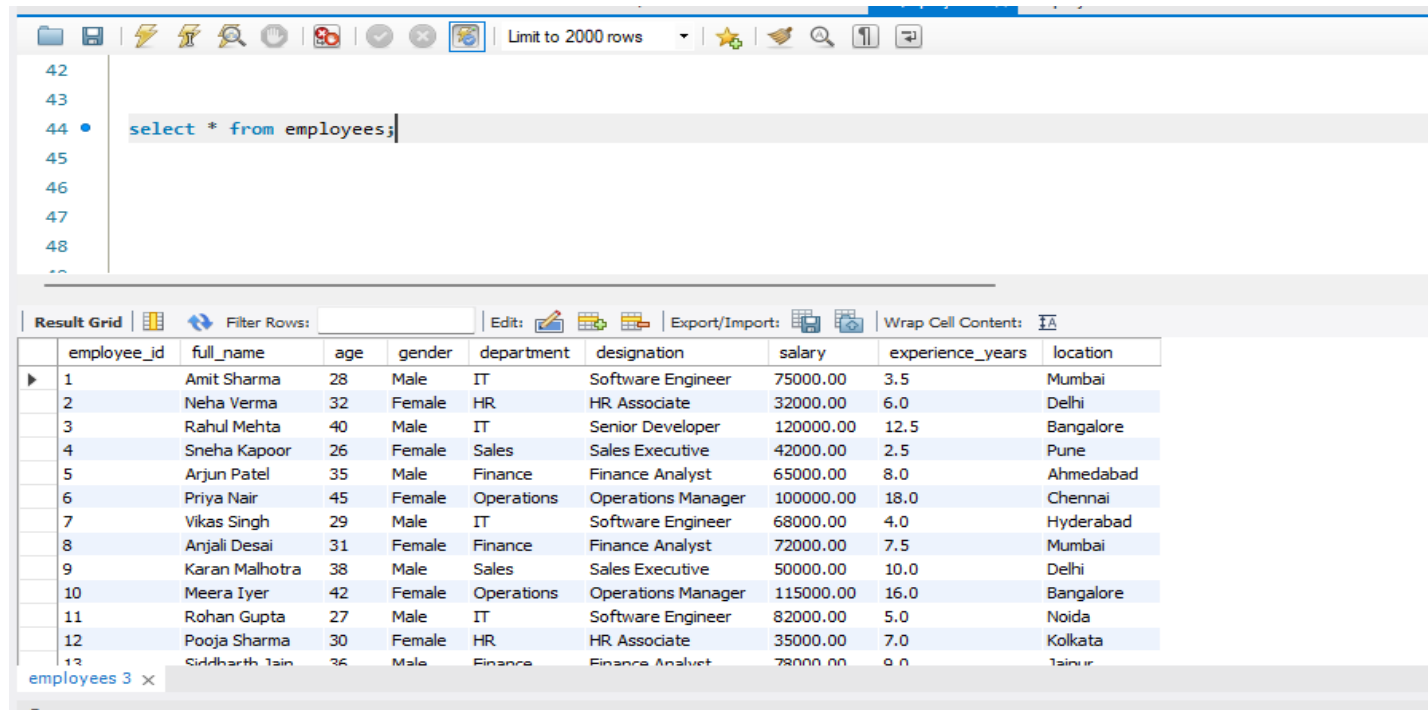
```

32 • select *,
33     case
34     when age between 22 and 30 then "Junior Employees"
35     when age between 31 and 40 then "Mid-Level Employees"
36     else "Senior employees"
37     end
38     as age_group from employees;

```

Result Grid										
Filter Rows:										
Export: Wrap Cell Content:										
	employee_id	full_name	age	gender	department	designation	salary	experience_years	location	age_group
▶	1	Amit Sharma	28	Male	IT	Software Engineer	75000.00	3.5	Mumbai	Junior Employees
	2	Neha Verma	32	Female	HR	HR Associate	32000.00	6.0	Delhi	Mid-Level Employees
	3	Rahul Mehta	40	Male	IT	Senior Developer	120000.00	12.5	Bangalore	Mid-Level Employees
	4	Sneha Kapoor	26	Female	Sales	Sales Executive	42000.00	2.5	Pune	Junior Employees
	5	Arjun Patel	35	Male	Finance	Finance Analyst	65000.00	8.0	Ahmedabad	Mid-Level Employees
	6	Priya Nair	45	Female	Operations	Operations Manager	100000.00	18.0	Chennai	Senior employees
	7	Vikas Singh	29	Male	IT	Software Engineer	68000.00	4.0	Hyderabad	Junior Employees
	8	Anjali Desai	31	Female	Finance	Finance Analyst	72000.00	7.5	Mumbai	Mid-Level Employees
	9	Karan Malhotra	38	Male	Sales	Sales Executive	50000.00	10.0	Delhi	Mid-Level Employees
	10	Meera Iyer	42	Female	Operations	Operations Manager	115000.00	16.0	Bangalore	Senior employees
	11	Rohan Gupta	27	Male	IT	Software Engineer	82000.00	5.0	Noida	Junior Employees
	12	Pooja Sharma	30	Female	HR	HR Associate	35000.00	7.0	Kolkata	Junior Employees
	13	Siddharth Jain	36	Male	Finance	Finance Analyst	78000.00	9.0	Jaipur	Mid-Level Employees

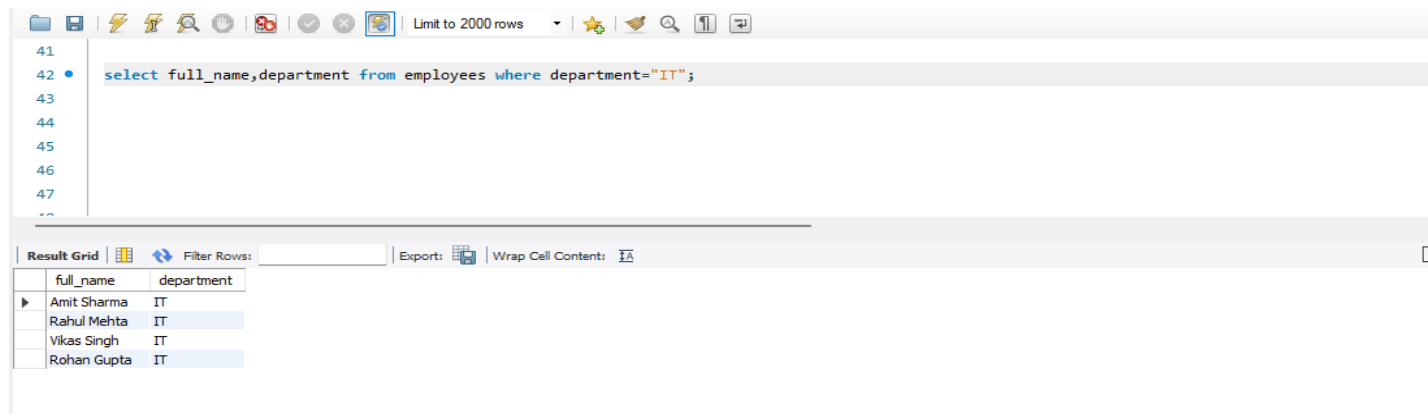
1. Retrieve all employee details.



The screenshot shows a database query tool interface. The top toolbar includes icons for file operations, a search icon, and a 'Limit to 2000 rows' dropdown. The SQL editor on the left contains the query: `select * from employees;`. The 'Result Grid' on the right displays the results of the query. The grid has columns for employee_id, full_name, age, gender, department, designation, salary, experience_years, and location. The results are listed in a table with 13 rows.

employee_id	full_name	age	gender	department	designation	salary	experience_years	location
1	Amit Sharma	28	Male	IT	Software Engineer	75000.00	3.5	Mumbai
2	Neha Verma	32	Female	HR	HR Associate	32000.00	6.0	Delhi
3	Rahul Mehta	40	Male	IT	Senior Developer	120000.00	12.5	Bangalore
4	Sneha Kapoor	26	Female	Sales	Sales Executive	42000.00	2.5	Pune
5	Arjun Patel	35	Male	Finance	Finance Analyst	65000.00	8.0	Ahmedabad
6	Priya Nair	45	Female	Operations	Operations Manager	100000.00	18.0	Chennai
7	Vikas Singh	29	Male	IT	Software Engineer	68000.00	4.0	Hyderabad
8	Anjali Desai	31	Female	Finance	Finance Analyst	72000.00	7.5	Mumbai
9	Karan Malhotra	38	Male	Sales	Sales Executive	50000.00	10.0	Delhi
10	Meera Iyer	42	Female	Operations	Operations Manager	115000.00	16.0	Bangalore
11	Rohan Gupta	27	Male	IT	Software Engineer	82000.00	5.0	Noida
12	Pooja Sharma	30	Female	HR	HR Associate	35000.00	7.0	Kolkata
13	Siddharth Jain	36	Male	Finance	Finance Analyst	78000.00	9.0	Jainpur

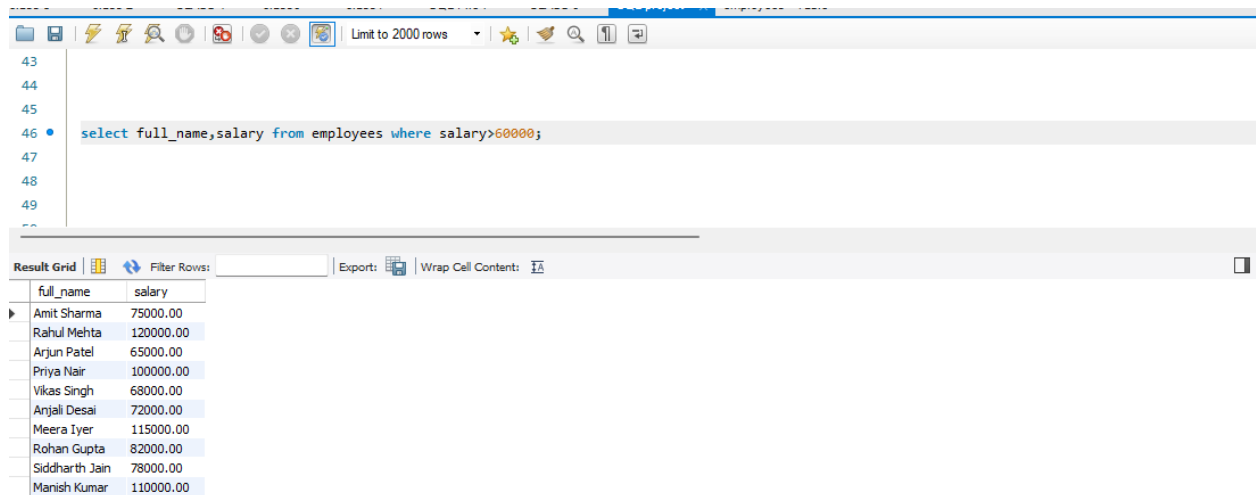
2. Find employees who work in the **IT** department.



The screenshot shows the same database query tool interface. The SQL editor on the left contains the query: `select full_name, department from employees where department="IT";`. The 'Result Grid' on the right displays the results of the filtered query. The grid has columns for full_name and department. The results are listed in a table with 4 rows, showing only employees from the IT department.

full_name	department
Amit Sharma	IT
Rahul Mehta	IT
Vikas Singh	IT
Rohan Gupta	IT

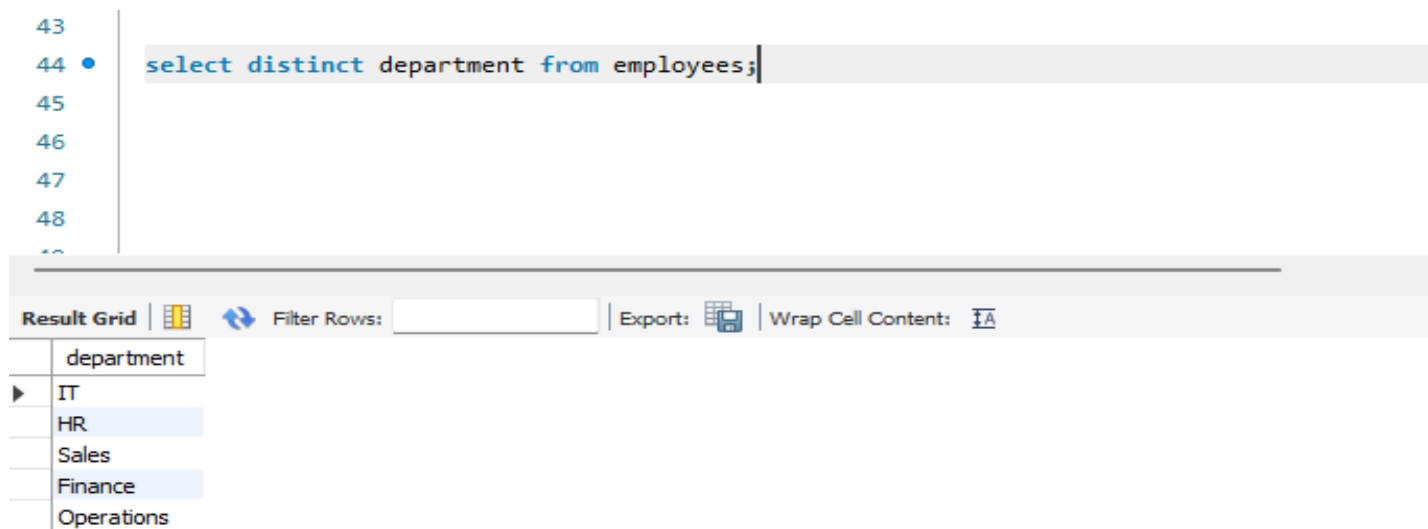
3. Show employees with salary **greater than 60,000**.



The screenshot shows a SQL IDE interface. At the top, there's a toolbar with various icons and a 'Limit to 2000 rows' dropdown. Below the toolbar, a SQL query is entered in a text area: `select full_name,salary from employees where salary>60000;`. The query is highlighted in blue. Below the query, the 'Result Grid' is displayed, showing a table with two columns: 'full_name' and 'salary'. The table contains 10 rows of data, with the first row expanded to show the full name 'Amit Sharma' and salary '75000.00'. The other rows are collapsed, showing only the first column 'full_name'.

full_name	salary
Amit Sharma	75000.00
Rahul Mehta	120000.00
Arjun Patel	65000.00
Priya Nair	100000.00
Vikas Singh	68000.00
Anjali Desai	72000.00
Meera Iyer	115000.00
Rohan Gupta	82000.00
Siddharth Jain	78000.00
Manish Kumar	110000.00

4. Get distinct department names.



The screenshot shows a SQL IDE interface. At the top, there's a toolbar with various icons and a 'Limit to 2000 rows' dropdown. Below the toolbar, a SQL query is entered in a text area: `select distinct department from employees;`. The query is highlighted in blue. Below the query, the 'Result Grid' is displayed, showing a table with one column: 'department'. The table contains 5 rows of data: 'IT', 'HR', 'Sales', 'Finance', and 'Operations'. The first row is expanded to show the full name 'IT'.

department
IT
HR
Sales
Finance
Operations

5. Find employees from **Mumbai** location.

```
46
47 • select * from employees where location="Mumbai";
```

[illegible]

6. Sort employees by **salary in descending order**.

```
48
49 • select * from employees order by salary desc;
```

Result Grid		Filter Rows:				Edit:		Export/Import:		Wrap Cell Content:	
	employee_id	full_name	age	gender	department	designation	salary	experience_years	location		
3		Rahul Mehta	40	Male	IT	Senior Developer	120000.00	12.5	Bangalore		
10		Meera Iyer	42	Female	Operations	Operations Manager	115000.00	16.0	Bangalore		
15		Manish Kumar	48	Male	Operations	Operations Manager	110000.00	20.0	Indore		
6		Priya Nair	45	Female	Operations	Operations Manager	100000.00	18.0	Chennai		
11		Rohan Gupta	27	Male	IT	Software Engineer	82000.00	5.0	Noida		
13		Siddharth Jain	36	Male	Finance	Finance Analyst	78000.00	9.0	Jaipur		
1		Amit Sharma	28	Male	IT	Software Engineer	75000.00	3.5	Mumbai		
8		Anjali Desai	31	Female	Finance	Finance Analyst	72000.00	7.5	Mumbai		
7		Vikas Singh	29	Male	IT	Software Engineer	68000.00	4.0	Hyderabad		
5		Arjun Patel	35	Male	Finance	Finance Analyst	65000.00	8.0	Ahmedabad		
9		Karan Malhotra	38	Male	Sales	Sales Executive	50000.00	10.0	Delhi		
14		Divya Reddy	33	Female	Sales	Sales Executive	47000.00	6.5	Chandigarh		
4		Shruti Kapoor	26	Female	Sales	Sales Executive	42000.00	2.5	Pune		

7. Show employees whose name starts with 'A'.

```
51 • select * from employees where full_name like "a%";
```

[illegible]

8. Find the **average salary** of each department.

```
53 • select department, round(avg(salary),0) as avg_sal from employees group by department;
```

department	avg_sal
IT	86250
HR	33500
Sales	46333
Finance	71667
Operations	108333

9. Find the department with the **highest number of employees**.

```
57 • with displ as (select department, count(employee_id) over (partition by department) as emp_count from employees)
58 select distinct department, emp_count from displ where emp_count=
59 (select max(emp_count) from displ);
```

department	emp_count
IT	4

10. Show **total salary paid** per department.

```
69 • select department, sum(salary) as tot_dept_sal from employees group by department;
```

department	tot_dept_sal
IT	345000.00
HR	67000.00
Sales	139000.00
Finance	215000.00
Operations	325000.00

11. List the **minimum and maximum salary** in the IT department.

```
70
71 • select department, min(salary) as min_sal,max(salary) as max_sal from employees where department="IT";
72
73
74
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	department	min_sal	max_sal	
IT		68000.00	120000.00	

12. Find employees with **more than 10 years** of experience.

```
72
73 • select * from employees where experience_years>10;
74
75
76
```

[illegible]

13. Categorize employees as **Junior/Mid/Senior level** using CASE based on age.

```
31
32 • select *,
33 case
34 when age between 22 and 30 then "Junior Employees"
35 when age between 31 and 40 then "Mid-Level Employees"
36 else "Senior employess"
37 end
38 as age_group from employees;
39
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	employee_id	full_name	age	gender	department	designation	salary	experience_years	location	age_group
▶	1	Amit Sharma	28	Male	IT	Software Engineer	75000.00	3.5	Mumbai	Junior Employees
	2	Neha Verma	32	Female	HR	HR Associate	32000.00	6.0	Delhi	Mid-Level Employees
	3	Rahul Mehta	40	Male	IT	Senior Developer	120000.00	12.5	Bangalore	Mid-Level Employees
	4	Sneha Kapoor	26	Female	Sales	Sales Executive	42000.00	2.5	Pune	Junior Employees
	5	Arjun Patel	35	Male	Finance	Finance Analyst	65000.00	8.0	Ahmedabad	Mid-Level Employees
	6	Priya Nair	45	Female	Operations	Operations Manager	100000.00	18.0	Chennai	Senior employess
	7	Vikas Singh	29	Male	IT	Software Engineer	68000.00	4.0	Hyderabad	Junior Employees
	8	Anjali Desai	31	Female	Finance	Finance Analyst	72000.00	7.5	Mumbai	Mid-Level Employees
	9	Karan Malhotra	38	Male	Sales	Sales Executive	50000.00	10.0	Delhi	Mid-Level Employees
	10	Meera Iyer	42	Female	Operations	Operations Manager	115000.00	16.0	Bangalore	Senior employess

Result 6 x | Read Only

14. Display salary range category based on designation.

```
76
77 • with displ1 as
78 (select *, min(salary) over(partition by designation) as min_sal,
79  max(salary) over(partition by designation) as max_sal from employees)
80 select *, concat(min_sal,"-",max_sal) as sal_range from displ1;
81
82
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	employee_id	full_name	age	gender	department	designation	salary	experience_years	location	min_sal	max_sal	sal_range
	5	Arjun Patel	35	Male	Finance	Finance Analyst	65000.00	8.0	Ahmedabad	65000.00	78000.00	65000.00-78000.00
	8	Anjali Desai	31	Female	Finance	Finance Analyst	72000.00	7.5	Mumbai	65000.00	78000.00	65000.00-78000.00
	13	Siddharth Jain	36	Male	Finance	Finance Analyst	78000.00	9.0	Jaipur	65000.00	78000.00	65000.00-78000.00
	2	Neha Verma	32	Female	HR	HR Associate	32000.00	6.0	Delhi	32000.00	35000.00	32000.00-35000.00
	12	Pooja Sharma	30	Female	HR	HR Associate	35000.00	7.0	Kolkata	32000.00	35000.00	32000.00-35000.00

Result 14 x | Read Only

15. Retrieve employees where salary is **between 50,000 and 1,00,000**.

```
80 select * from employees where salary between 50000 and 100000;
81
82 •
83
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

employee_id	full_name	age	gender	department	designation	salary	experience_years	location
1	Amit Sharma	28	Male	IT	Software Engineer	75000.00	3.5	Mumbai
5	Arjun Patel	35	Male	Finance	Finance Analyst	65000.00	8.0	Ahmedabad
6	Priya Nair	45	Female	Operations	Operations Manager	100000.00	18.0	Chennai
7	Vikas Singh	29	Male	IT	Software Engineer	68000.00	4.0	Hyderabad
8	Anjali Desai	31	Female	Finance	Finance Analyst	72000.00	7.5	Mumbai

employees 15 x

Output

Action Output

16. Find all **female employees** in the Sales department.

```
83
84 • select * from employees where department="Sales" and gender="Female";
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

employee_id	full_name	age	gender	department	designation	salary	experience_years	location
4	Sneha Kapoor	26	Female	Sales	Sales Executive	42000.00	2.5	Pune
14	Divya Reddy	33	Female	Sales	Sales Executive	47000.00	6.5	Chandigarh
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

17. Retrieve employees whose experience is **between 2 and 8 years**.

```
85
86
87 • select * from employees where experience_years between 2 and 8;
88
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

employee_id	full_name	age	gender	department	designation	salary	experience_years	location
1	Amit Sharma	28	Male	IT	Software Engineer	75000.00	3.5	Mumbai
2	Neha Verma	32	Female	HR	HR Associate	32000.00	6.0	Delhi
4	Sneha Kapoor	26	Female	Sales	Sales Executive	42000.00	2.5	Pune
5	Arjun Patel	35	Male	Finance	Finance Analyst	65000.00	8.0	Ahmedabad
7	Vikas Singh	29	Male	IT	Software Engineer	68000.00	4.0	Hyderabad

employees 17 x

18. Create a self join to compare employees with **higher salary** vs lower salary.

```
88
89 • select * from employees;
90 • select l.full_name as low_paid_emp, l.salary as lower_salary,
91        h.full_name as high_paid_emp, h.salary as higher_sal
92        from employees as l inner join employees as h on l.salary<h.salary;
93
94
95
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

low_paid_emp	lower_salary	high_paid_emp	higher_sal
Divya Reddy	47000.00	Amit Sharma	75000.00
Pooja Sharma	35000.00	Amit Sharma	75000.00
Karan Malhotra	50000.00	Amit Sharma	75000.00
Anjali Desai	72000.00	Amit Sharma	75000.00
Vikas Singh	68000.00	Amit Sharma	75000.00

Result 4 x

Read Only

19. Create a view high_salary_employees showing employees who earn more than 80,000.

```
93
94 • create view t1 as (select * from employees where salary>80000);
95
96 • select *from t1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

employee_id	full_name	age	gender	department	designation	salary	experience_years	location
3	Rahul Mehta	40	Male	IT	Senior Developer	120000.00	12.5	Bangalore
6	Priya Nair	45	Female	Operations	Operations Manager	100000.00	18.0	Chennai
10	Meera Iyer	42	Female	Operations	Operations Manager	115000.00	16.0	Bangalore
11	Rohan Gupta	27	Male	IT	Software Engineer	82000.00	5.0	Noida
15	Manish Kumar	48	Male	Operations	Operations Manager	110000.00	20.0	Indore

t1 5 x

Read Only

20. Query the view to list employees with more than 5 years experience.

```
97
98 • select * from t1 where experience_years>5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

employee_id	full_name	age	gender	department	designation	salary	experience_years	location
3	Rahul Mehta	40	Male	IT	Senior Developer	120000.00	12.5	Bangalore
6	Priya Nair	45	Female	Operations	Operations Manager	100000.00	18.0	Chennai
10	Meera Iyer	42	Female	Operations	Operations Manager	115000.00	16.0	Bangalore
15	Manish Kumar	48	Male	Operations	Operations Manager	110000.00	20.0	Indore

21. Rank employees by salary (highest to lowest).

.00

.01 • `select *,dense_rank() over(order by salary desc) as rnk from employees;`

.02

.03

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

employee_id	full_name	age	gender	department	designation	salary	experience_years	location	rnk
3	Rahul Mehta	40	Male	IT	Senior Developer	120000.00	12.5	Bangalore	1
10	Meera Iyer	42	Female	Operations	Operations Manager	115000.00	16.0	Bangalore	2
15	Manish Kumar	48	Male	Operations	Operations Manager	110000.00	20.0	Indore	3
6	Priya Nair	45	Female	Operations	Operations Manager	100000.00	18.0	Chennai	4
11	Rohan Gupta	27	Male	IT	Software Engineer	82000.00	5.0	Noida	5

Result 12

22. Display each employee's salary along with **department-wise average salary**.

102

103 • `select full_name, department, salary, round(avg(salary) over(partition by department),0) as avg_sal from employees;`

104

105

106

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
full_name	department	salary	avg_sal
Arjun Patel	Finance	65000.00	71667
Anjali Desai	Finance	72000.00	71667
Siddharth Jain	Finance	78000.00	71667
Neha Verma	HR	32000.00	33500
Pooja Sharma	HR	35000.00	33500

23. For each department, calculate **dense_rank** of employees based on experience.

104

105 • `select full_name, department, experience_years, dense_rank() over(partition by department order by experience_years) as rnk from employees;`

106

107

108

Result Grid   Filter Rows: Export:  Wrap Cell Content: 

	full_name	department	experience_years	rnk
▶	Anjali Desai	Finance	7.5	1
	Arjun Patel	Finance	8.0	2
	Siddharth Jain	Finance	9.0	3
	Neha Verma	HR	6.0	1
	Pooja Sharma	HR	7.0	2

Result 21 ×

 Read On

Output

 Action Output